

UNIVERSITY OF MICHIGAN
Department of Electrical Engineering and Computer Science
EECS 445 — Introduction to Machine Learning
Fall 2025

Project 2: Paco’s Puppy Problem

An exploration of deep learning techniques for classification and representation learning

Due: Wednesday, 11/5 at 10:00pm

Section	Points	Recommended Completion Date
1. Data Preprocessing	10	Thursday, 10/23
2. Convolutional Neural Networks	30	Saturday, 10/25
3. Transfer Learning	18	Tuesday, 10/28
4. Vision Transformers	22	Saturday, 11/1
5. Challenge	20	Wednesday, 11/5
Total	100	

Introduction

Dog-gone! The neighborhood kennel owner, Paco, is having a “ruff” time: during his morning rounds to check on his new foster puppies, he found to his horror that they had all dug their way under the fence and escaped! He’s now enlisted the help of the entire neighborhood to help round up his lost pups. He’s asked his neighbors to send in photos of dogs that they believe could have escaped from his kennel. Now Paco is receiving thousands of images a day—too many to go through himself. He needs your help to build a system to automatically distinguish his dogs from the others.

Paco’s kennel houses Golden Retrievers, but Cyrus’s kennel that houses Collies has experienced an escape of its own! We need a classifier that distinguishes Paco’s Golden Retrievers versus Cyrus’s Collies. To help with that, Paco has provided us with a few images of his dogs along with images of Collies. He has also sent images of other breeds including Samoyeds, Miniature Poodles, Great Danes, Dalmatians, Siberian Huskies, Yorkshire Terriers, Chihuahuas, and Saint Bernards just in case those might be helpful. We’ve included some of the pictures he sent below.



Figure 1: Sample images from the the dataset.

In our mission to help Paco, we will explore supervised deep learning techniques for image data. In this setting, we will use a convolutional neural network (CNN) to classify Paco’s dog images by breed, where the breed associated with each image serves as the classification label. We will then explore *supervised pretraining*, a type of *transfer learning* that will use a feature representation learned on a related task to augment our classification abilities. Finally, we will also try out a new architecture—Vision Transformers (ViT)—that borrows ideas from the original transformer architecture, which is used for language tasks. After this, you will be well-equipped to take on Paco’s challenge: implementing and training a deep neural network of your own design to classify dog images by breed!

Getting Started

This project will make extensive use of [PyTorch](#), an open source AI framework. Coding questions will be highlighted **red** whereas questions that require written answers will be highlighted **yellow**.

Please download the Project 2 skeleton code `project2.zip` from Canvas. This zip file contains the full project dataset, and may take several minutes to download.

Requirements

You should have Python 3.13.3 and Anaconda installed from previous assignments. If you do not, please refer to the course’s python setup guide on Canvas which can be found under files at [Tutorial/Python Setup Guide.pdf](#).

Run the following command to create a conda environment and install necessary packages:

```
conda create --name 445_p2 --file requirements.txt
```

Common Install Issues

If you are using **WSL**, some [extra configuration](#) may be necessary for matplotlib to correctly display the plots. If you’re using **linux**, you may need to install [PyQt5](#) as well. However, all plots will always be saved to files as well as displayed, so it’s not strictly necessary to fix visual issues.

If you are using an **Intel Mac**, you may need to install older, non-deprecated versions of `pytorch` and `torchvision`. To do this, replace `pytorch==2.6.0` with just `pytorch` (and the same for `torchvision`) in `requirements.txt`.

Compute

We have written this project to run in a reasonable time on a standard laptop; however, if you have access to a compatible GPU and wish to use it, you can [manually install PyTorch](#) with the corresponding CUDA version. Note that you will have to use `pip` instead of `conda` to create your virtual environment and install the requirements file when using a GPU. To ensure fairness, challenge submissions that utilize GPU hardware will be graded separately from non-GPU submissions.

Dataset

This dataset contains 8,867 PNG image files of 10 different dog breeds. These images are stored under `data/images/`. Each image contains 3 color channels (red, green, blue) and is of size $3 \times 64 \times 64$. These images have already been divided into 4 data partitions: a training set, a validation set, a test set and a held-out challenge set. Metadata containing the image label and information regarding to what data partition the image belongs is documented in `data/dogs.csv`. Note that the challenge labels have been removed from this file. As in Project 1, you will include your challenge model's predictions on this data partition as a part of your submission.

1 Data Preprocessing [10 pts]

Real datasets often contain numerous imperfections (e.g., incorrect labels, feature noise, or irrelevant examples) and may not be immediately useful for training our model. To develop a dataset more conducive for training, we will explore normalizing our data. The preprocessing will be implemented in the file `dataset.py`.

- (a) **Implement** the `fit()` and `transform()` functions in the `ImageStandardizer` class (defined in `dataset.py`).
- The `fit()` function is first called with the training dataset as input, and it calculates and stores the per-channel mean and standard deviation from the training data. After running this function, you will have a mean and standard deviation for the R channel, G channel, and B channel (6 scalars total for the entire training data). These mean and standard deviation values are computed from training data.
 - The `transform()` function then applies the values computed and stored in `fit()` to the standardization of all data partitions (validation/testing). Specifically, we assume that the underlying distributions of each image channel (red, green, and blue) are independent distributions with mean 0 and variance 1. To embed these assumptions, zero-center each image channel by subtracting the per-channel mean value, then scale each image channel by dividing by the per-channel standard deviation.

You may use the provided `test.ImageStandardizer.py` file to test your implementations.

Note: When reading in images, make sure images are represented as `np.float64` rather than `ints`.

Answer the following questions regarding this normalization process.

- (4 points) Run the script `dataset.py`. What is the mean and standard deviation of each color channel (RGB), learned from the entire training partition?
 - (4 points) Why do we extract the mean and standard deviation from the training set as opposed to the other data partitions?
- (b) (2 points) **Run** the script `visualize_data.py`¹ to check that the preprocessing did not result in a loss of information. If you did it correctly, the images should look slightly blurred after running `visualize_data.py`. **Include one image generated** from `visualize_data.py` in your writeup. This should be a grid showing four random images and then their preprocessed versions.

¹The script samples 4 random images from the dataset for each of original and preprocessed. Click anywhere on the figure to randomly sample from the dataset again, and close the figure to terminate the program.

2 Convolutional Neural Networks [30 pts]

With our data now in a suitable form for training, we will explore the implementation and effectiveness of a convolution neural network (CNN) on the dataset. In this section, we train a CNN to distinguish between **2 classes: Collies and Golden Retrievers** (while our dataset has more classes, we will ignore them here and explore them more in Section 3). We will call this model the Target model. Recall the basic structure of a CNN: a sequential combination of convolutional, activation, pooling and fully connected layers.

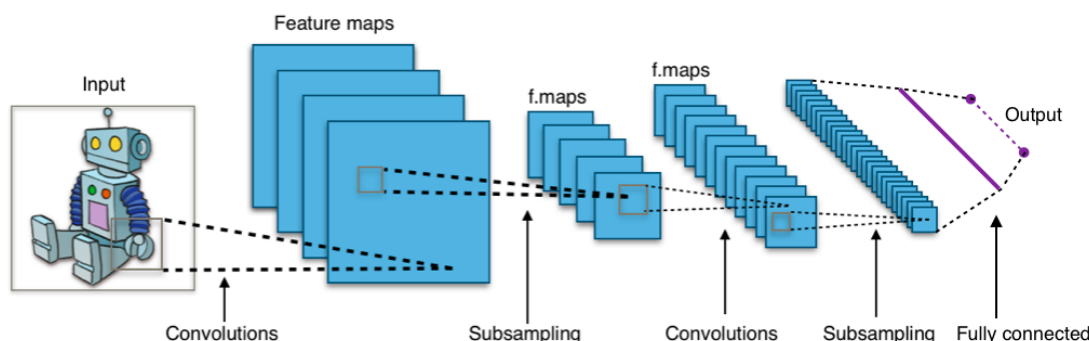


Figure 2: An example convolutional neural network. By Aphex34, licensed under [CC BY-SA 4.0](#).

CNNs used for image classification can often be viewed in two parts, where the first part learns a representation of the image data—or “encoding”—and the second learns a mapping from this image encoding to the output space. Our CNN will follow this schema, and the output space will be the two class labels we are considering. The image encoder will be composed of convolutional layers followed by max pooling (also called subsampling), which captures patterns across the image data by sliding filters with learned weights across the image. Once we have our image encoding, we will train a single fully connected layer to learn a linear combination of this representation, from which we will ultimately derive our classification.

We have provided the training framework necessary to complete this problem in the file `train_cnn.py`. All of the following problems that involve implementation will require you to use the PyTorch API. We’ve provided some tips in Appendix A, and you are encouraged to explore the [documentation](#) and [online tutorials](#) to learn more.

- (3 points) **Study** the Target architecture specified in Appendix B. **How many learnable float-valued parameters does the model have?** Show your work by manually calculating this number, and you can later verify against the program output after implementing the model.
- Implement** the architecture by completing the `Target` class in `model/target.py`. In pytorch, we define a neural net by subclassing `torch.nn.Module`, which handles all the gradient computations (back propagation) automatically. Your job is to fill in the appropriate lines in three functions: `__init__()`, `init_weights()` and `forward(x)`.
 - Use `__init__()` to define the architecture, i.e. what layers our network contains. At the end of `__init__()` we call `init_weights()` to initialize all model parameters (weights and biases) in all layers to desired distributions. Appendix A may provide tips to implement this function.

- The `forward(x)` function defines the forward propagation for a batch of input examples, by successively passing output of the previous layer as the input into the next layer after applying activation functions, and returning the final output as a `torch.Tensor` object.
 - The `torch.Tensor` class implements a `backward()` function. As its name suggests, it performs back propagation and computes the partial derivatives with respect to each model parameter using chain rule. While you do not have to implement `backward()`, you should review the lecture slides on backpropagation, so that you understand why it is important when training the network.
- (c) After making a forward pass, we can use the output from the CNN to make a prediction. **Implement** the `predictions()` function in `train_common.py` to predict the numeric class label given the network output. This output is just the raw output prediction of the network (without normalization). *Hint: When interpreting the raw output from the model, think about what the **true** labels are and note that the network is trying to output this true label. Pay attention to the fact that the output dimension is 2 and think about how the indices of the output vector relate to the class labels.*
- (d) **Review** the training script `train_cnn.py`. Based on the model specification in Appendix B, **fill in** the definitions for `criterion` and `optimizer`.
- (e) In `train_common.py`, `train_epoch()` loops through the training data and updates the model parameters based on the criterion and optimizer. After a complete pass of the training set (called an “epoch”), we evaluate the model on the validation set, and save this solution as a checkpoint. We can then use `evaluate_epoch` to evaluate the learned parameters on both the training and validation data. **Review** the definitions of the following functions:
- `train_epoch()`: within one epoch, we pass batches of training examples through the network, use backpropagation to compute gradients, and update model weights using the gradients.
 - `evaluate_epoch()`: we pass the entire validation set (in batches) through the network and get the model’s predictions, and compare these with the true labels to get an evaluation metric.
 - `save_checkpoint()`: checkpointing—the periodic saving of model parameters—is an important technique for training large models in machine learning; if a hardware failure occurs due to a power outage or our code fails for whatever reason, we don’t want to lose all of our progress!

Given the size of our training data, we’re unlikely to have learned a good model after a single epoch. Thus, we will have to loop over the data multiple times i.e., train for multiple epochs. But how will we know when to stop? We will use ‘early stopping’ and leverage the performance on the validation data. More specifically, we will stop training after the validation loss does not strictly decrease for some number of epochs defined with the variable `patience`. In other words, we should keep track of how many contiguous epochs have passed in which the validation loss has either stayed the same or increased. See `log_training` in `utils.py` for how to extract this validation loss from the stats data structure. In `train_common.py` **implement** `early_stopping` and **implement** the training steps in `train_epoch`.

- (f) **Execute** `train_cnn.py` to train the model with a patience of 5.

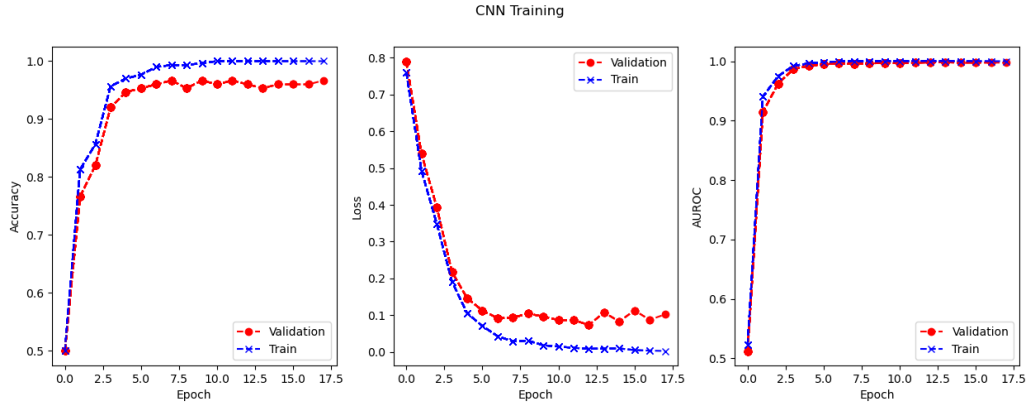


Figure 3: Example training plot for the target task.

Note that if you run the script more than once, you will be prompted to enter the epoch number at which the model parameters from the saved checkpoint will be restored and training will *continue* from that epoch. If you enter 0, then the model will be trained from scratch and the old checkpoints will be deleted.

This script will also create three graphs that are updated every epoch to monitor the model's performance in terms of loss, accuracy, and AUROC on the train and validation set. **Answer** the following questions.

- i. (4 points) You may observe that the validation loss does not monotonically decrease in the training plot. It should look similar to Figure 3. Note that getting a different number of epochs in your plots or having slightly different trends does not mean that your code implementation is wrong. **Describe** at least two sources of noise that contribute to this behavior.
- ii. (6 points) **Report** the epoch that your model stopped training at with using a patience of 5. Try increasing your patience to 10. Make sure to load from epoch 0 for a fresh model. **Answer** the following: What epoch does your model stop training at now? Based on the training graphs, which patience value works better and why? In what scenario might increased patience be better? **Include** the training plot generated by using a patience of 5 and the one with a patience of 10 in your answer.
- iii. (9 points) Let's explore how changing our model architecture affects performance. **Change** the number of output filters in the last convolutional layer from 8 to 64 and update the input dimension of the fully connected layer accordingly. What's the new size of the input to the fully connected layer? **Report** the new size and update the architecture in `model/target.py`. Using a patience of 5, train the model again with the updated architecture starting from epoch 0. **Complete** the table below, reporting the area under the receiver operating characteristic (AUROC) curve on the validation and training set for the model (i.e., epoch) **associated with the lowest validation loss** (note: this is not necessarily the model associated with the last epoch). How does performance vary as we increase the number of filters from 8 to 64? What might account for this change? **Answer** these questions in a few sentences.

	Epoch	Training AUROC	Validation AUROC
8 filters			
64 filters			

(g) **Execute** `test_cnn.py` to measure the performance of your trained CNN on the heldout test set. `test_cnn.py` measures the performance of your model using both accuracy and AUROC scores on the training data, validation data, and test data. The model being evaluated in this section will be our original trained CNN from Section 2(f) with a patience of 5 and 8 filters in the last convolutional layer. You will need to train a new CNN (execute `train_cnn.py` with epoch 0) with this architecture if the most recent model you trained doesn't match these specifications. When running `test_cnn.py`, make sure to load your trained model using the optimal number of epochs you chose in Section 2(f)iii.

- i. (3 points) **Include** your CNN's scores in the table below.

	Training	Validation	Testing
Accuracy			
AUROC			

- ii. (2 points) We know that if our training performance far exceeds our validation performance it means that we may have overfit to the training data. **Compare** the training and validation performance. Do we see any evidence of overfitting here?
- iii. (3 points) **Run** the script `visualize_labels.py`². What do you notice about the images of the positive labeled data compared to the negative labeled data? Then, **compare** the validation and test performance. What trend do you see here and how could it be related to the data? Hypothesize a possible explanation for such a trend.

²The script samples 5 random images from the dataset for each label. Click anywhere on the figure to randomly sample from the dataset again, and close the figure to terminate the program.

3 Transfer Learning [18 pts]

From our results in Section 2(g), there appears to be something up with the data. Oh no! In this section we will explore a strategy that could help us improve our performance on the test set: transfer learning.

In settings with only limited training data, we may benefit from leveraging auxiliary data that isn't directly involved with our task. For example, suppose we want to train a model to classify cars versus bicycles. Let's say we only have a limited amount of labeled car and bicycle images, but a large auxiliary dataset of trucks versus pedestrians. In such a scenario, we refer to the task of interest as the *target* task and the auxiliary task as the *source* task.

When the target and source tasks are related, we might seek to *transfer* knowledge about the source task to the target task. For example, the features useful for distinguishing trucks from people may also be useful in distinguishing cars from bicycles (e.g., number of wheels, placement on road). There are multiple ways to take advantage of the source model when training the target model (i.e. to perform transfer learning). Here are some:

- Initialize the target model weights with the source model's pre-learned weights and train all layers of the target model.
- Keep the first k layers of the source model and train the last $L-k$ layers of the target model using the target dataset (where L is the number of layers in the source model).
- Keep all layers of the source model except the last and fine-tune the last layer of the model using the target dataset. This is essentially learning a “feature extractor” on the source task and applying it to the target task to learn how these features combine to make target predictions.

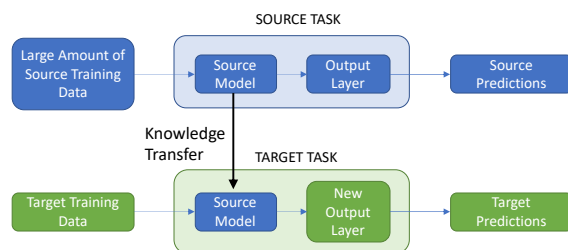


Figure 4: In settings with limited training data we may be able to transfer knowledge from a related but distinct source task with large amounts of training data.

In this question, our target task is the binary classification task of distinguishing Collies from Golden Retrievers. However, we have labeled training data from 8 other breeds: Samoyed, Miniature Poodle, Saint Bernard, Great Dane, Dalmatian, Chihuahua, Siberian Husky, and Yorkshire Terrier. We hypothesize that the learned representations, or features, useful for solving this source task will transfer to our target task and improve our model's performance on target predictions. In the steps below, we will explore a transfer learning approach to leveraging these source data.

- (a) Study the architectures presented in Appendix B and Appendix C. These architectures will correspond to the architecture of the target and source models.

Implement the Source model architecture by completing the `Source` class in `model/source.py` based on the model architecture specifications in Appendix C. Follow the same steps as implementation for the `Target` from Section 2(b). **Be sure not to change** the name of fully connected layers from the original code in `source.py` and `target.py`. The names of these layers are intentionally different to work with the way that PyTorch restores model parameters from checkpoints.

- (b) **Review** the script `train_source.py`. Based on the model specification in Appendix C, **fill in** the definitions for `criterion` and `optimizer`.
- (c) (6 points) **Run** the script `train_source.py` to train the network with a patience of **10** on the 8 classes of training data and evaluate on the validation set. On a standard laptop, this might take a few minutes. **Include** the final training plot. Select an epoch number corresponding to the model with the lowest validation loss. This is the model we will transfer to our target task. **Report** the epoch number and use this value for the rest of the transfer learning questions.
- (d) (4 points) **Run** the script `confusion_matrix.py` to visualize the prediction accuracy of each class on the validation dataset. **Include** the plot in your write up. **Examine** for which breed(s) the classifier is the most accurate. For which breed(s) is the classifier the least accurate? **Briefly hypothesize** why this might be the case. Note that there is no need to report or calculate exact accuracy metrics for any breeds.
- (e) In Section 2, we randomly initialized the weights of the CNN. Here, we will leverage the model learned in Section 3(c) to initialize the weights. We can then selectively freeze or fine-tune different parts of our network (e.g., freezing all convolutional layers, freezing only the first two convolutional layers, etc.).

Review the script `train_target.py`. Based on the model specification in Appendix B, **fill in** the definitions for `criterion`, `optimizer`, and `patience`. **Implement** the `freeze_layers` function to modify the model with the given number of layers frozen. Note that Python is pass-by-reference, so modifying the model parameters also modifies the original model. Here, you must loop through `model.named_parameters()` to determine which convolutional layers to freeze.

- (f) (8 points) **Run** the script `train_target.py` with different number of convolutional layers frozen to train our classifier to fit the binary task. **Compare** this classifier to your previous CNN classifier in terms of the train, validation and test AUROC for the model associated with the lowest validation loss. Format your results as follows:

	AUROC		
	TRAIN	VAL	TEST
Freeze all CONV layers (Fine-tune FC layer)			
Freeze first two CONV layers (Fine-tune last CONV and FC layers)			
Freeze first CONV layer (Fine-tune last 2 conv. and fc layers)			
Freeze no layers (Fine-tune all layers)			
No Pretraining or Transfer Learning (Section 2(g) performance)			

Examine the results of using transfer learning and answer the following questions. Compared to the CNN in Section 2(g) that did not leverage the source task data, does transfer learning help? Was the source task helpful? Why do you hypothesize it was (or wasn't)? Examine the training curves. How does freezing all convolutional layers compare to freezing just a subset, versus freezing none? Why do you think this might occur?

4 Vision Transformers [22 points]

While we give a brief overview of the vision transformer (ViT) architecture here, refer to Appendix D for a more complete description of ViTs and how they differ from CNNs.

Now that we have explored using CNNs for image classification, let's explore using a *transformer* neural network architecture. To perform image classification tasks with a transformer, you will finish implementing the forward pass function of a ViT as proposed by [Dosovitskiy et al., 2021](#). Although the original transformer architecture by [Vaswani et al., 2017](#) was designed with natural language processing in mind, ViTs were proposed in an attempt to leverage the success of attention mechanisms in the domain of computer vision. Since proposed, ViTs have demonstrated to be effective at a variety of image-based tasks, often outperforming traditional CNNs.

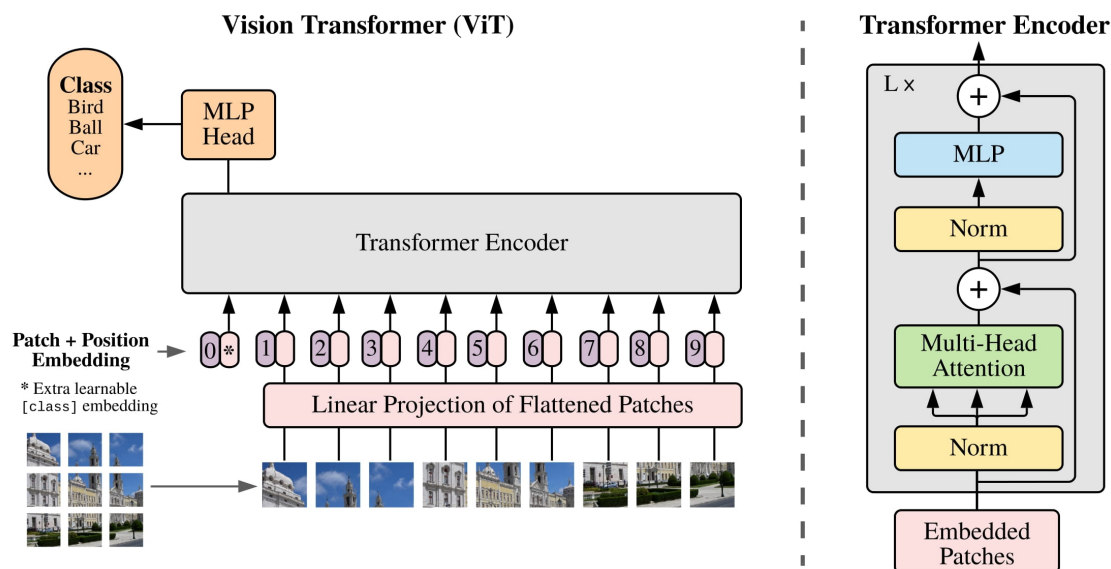


Figure 5: An illustration of the ViT architecture proposed by [Dosovitskiy et al., 2021](#).

- (a) (2 points) In the ViT architecture in Figure 5, we include an extra learnable [cls] embedding. Without such a learnable embedding how could you use the output of the transformer encoder to generate a classification? (Hint: Sections 3.1 and D.3 of [Dosovitskiy et al., 2021](#) may help understand the relevant ViT architecture. This technique was introduced in the BERT model, whose embedding system is explained in section 3, “Input/Output Representations,” of [Devlin et al., 2017](#).) Explain your reasoning.
- (b) (2 points) If the position embedding was omitted as a part of the input to the transformer encoder, what effect would this have on the model’s ability to classify images? Explain your reasoning.

- (c) In `models/vit.py`, implement the missing scaled dot product attention calculation, a key component of the `MultiHeadAttention` class's `forward()` function. The formula for scaled dot product attention in both matrix and vector form is provided here. Q , K , and V are matrices, where each row represents the query, key, and value vectors, respectively, for each token within the context window. $\bar{q}_i$, $\bar{k}_j$, and $\bar{v}_k$ are the query, key, and value vectors that, respectively, make up the rows of Q , K , and V . d is the number of dimensions of the query and the key vectors, sometimes called the “query size.”

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d}} \right) V$$

Scaled dot-product attention can also be viewed in vectorized form. The attention calculation for a single query-key-value triplet can be written as

$$\text{Attention}(\bar{q}_i, \bar{k}_j, \bar{v}_k) = \text{softmax} \left(\frac{\bar{q}_i \cdot \bar{k}_j}{\sqrt{d}} \right) \cdot \bar{v}_k.$$

- (d) In `models/vit.py`, implement the `TransformerEncoder` class's `forward()` function. The forward pass should match the Transformer Encoder scheme illustrated in the right column of Figure 5.
- (e) In `models/vit.py`, implement the two missing pieces of the `ViT` class's `forward()` function. Understanding the ViT architecture in Figure 5 along with the rest of the classes defined in `models/vit.py` may help you finish implementing the `forward()` function.
- (f) In this sub-question we will solidify your understanding of each component of the ViT by counting the number of learnable parameters of the model. We use the configurations in Appendix D. For each component, calculate the number of learnable parameters and show your work.
- i. (1 point) How many learnable parameters do we need for the `[cls]` token?
 - ii. (2 points) How many learnable parameters are there involved with the Patch Linear Projection Layer and Position Embedding (excluding the `[cls]` token)? (**Hint:** For patch embedding, we flatten the image and project it to our embedding dimension specified in Appendix D.)
 - iii. Now we will calculate the number of parameters in a single transformer block.
 1. (1 point) How many learnable parameters are there in a single LayerNorm layer?
 2. (1 point) How many learnable parameters are there in a single MHA layer?
 3. (1 point) How many learnable parameters are there in a single MLP layer?
 - iv. (1 point) How many learnable parameters are there in the final MLP head?
 - v. (1 point) Using your previous answers and Appendix D, how many total learnable parameters are there in our ViT? (**Hint:** you can verify your answers against the number of learnable parameters in `train_vit.py`.)
- (g) Review the training script `train_vit.py` which follows the same general format as `train_cnn.py`. Based on the model specification in Appendix D, fill in the definitions for `criterion`, `optimizer`, and `model` as you did previously for the CNN training script.

- (h) (4 points) **Execute** `train_vit.py` to train the ViT model on the target task from before with a patience of **5**. Remember to **set the `patience` and `curr_patience` to appropriate values**. **Include** the training figures generated by the script in your writeup as you did in earlier parts. **Report** the train/validation AUROC performance of the model from the epoch demonstrating the lowest validation loss.
- (i) (2 points) Next, using the model checkpoint saved from the epoch demonstrating the lowest validation loss, report the model's AUROC performance on the heldout test set by executing the `test_vit.py` script and selecting the appropriate epoch to load from. **Report** the saved model's AUROC performance on the test set.
- (j) (4 points) How does the chosen ViT model's validation and test performance compare to the best chosen CNN validation and test performance on the target task from Section 2(g)? How does the number of learnable parameters in the ViT compare to the number of learnable parameters in the CNN? **Explain why you believe they do or do not differ**.

5 Challenge [20 pts]

Armed with your knowledge of neural networks and PyTorch, you are now ready to take on Paco's challenge: **designing**, **implementing**, and **training** a deep neural network of your own design to classify dog breeds! We will again restrict our final classifier to Collies and Golden Retrievers; however, you are allowed to use the training and validation sets for all 10 classes to train your model. These additional data may be useful if you wish to further explore transfer learning in your challenge solution. In addition, you will have *full control* over the network architecture, including weight initialization, filter sizes, activation function, network depth, regularization techniques, preprocessing methods, and any other changes you see fit. You do not have to respect the current train/validation/test split, but are allowed to train any data that we give you, such as the validation and test data. You will be evaluated on a heldout challenge set drawn from the same distribution as the test set used previously in the project. We ask you to adhere to the following guidelines when developing your challenge submission.

- You *must* use only the data provided for this challenge; **do not attempt to acquire any additional training data, or modify the ground truth test labels.** However, you can apply data augmentation.
- While using the architecture of an existing model is ok, you must train them yourself. **You are not allowed to use pre-trained models and weight initializations.**
- **Implement** your model within `model/challenge.py`.
- **Fill in** the definitions for the loss function, optimizer, and model in `train_challenge.py`.
- You may alter the training framework in `train_challenge.py` as you please.
- If you wish to modify any portion of the data batching or preprocessing, please make a copy of `dataset.py` and call it `dataset_challenge.py`. The same applies for any other files you wish to modify such as `train_common.py` and `utils.py`. You may then modify this file however you choose.
- You may add additional variables to the `config.json` file and modify any of the existing challenge variables as you see fit.

Once your model has been trained, ensure that its checkpoint exists in the directory `checkpoints/challenge/` and **run** the script `predict_challenge.py` as follows.

```
python predict_challenge.py --username=<your_username> [--gpu]
```

This script will load your model from your saved checkpoint and produce the file `<your_username>.csv` that contains your model's predictions on the test set. **If you used a GPU to train your model, specify the `--gpu` flag after the `username` flag. Otherwise, do not add it.**

Double check that your final output csv file has float-value predictions (not binary 0/1 labelling) for each of the 200 images in the challenge dataset before you submit to gradescope. We've also provided a script `test_output.py` to test your output format.

```
python test_output.py <your_csv_file>
```

In your write-up, **describe in detail the experiments you conducted and how they influenced your final model.** When grading your challenge write-up, we will look for discussion of your decisions

regarding the following. If you chose not to implement some of the approaches, add a short discussion on methods you potentially could have done.

Here are some ideas:

- Regularization (weight decay, dropout, etc.)
- Data Augmentation
- Model architecture
- Hyperparameters
- Transfer learning
- Model evaluation (i.e., the criteria you used to determine which model is best)

We will evaluate your submission based on two components:

- (12 points) Effort: We will evaluate how much effort you have applied to this problem based on the experiments described in your write-up and your code submission.
- (8 points) AUROC: We will evaluate the AUROC of your classifier's predictions on the ground truth challenge labels. Note that this means that the output test predictions follow a specified order. For that reason, please **do not shuffle the challenge data points during prediction.**

REMEMBER to submit your project report by 10:00pm ET on November 5th, 2025 to Gradescope.

- **Upload** your write up to “Project 2 Writeup” on gradescope.
- **Upload** your entire project as a zip file to “Project 2 Code Appendix” on gradescope. You will receive a 0 if you do not do so. Make sure you are in the root directory of the project, and use the command below:

```
zip -r submission.zip . -x "./data/*" "./checkpoints/*" "./git/*"
```

Then, submit `submission.zip`.

- **Upload** your file `<username>.csv` containing the label predictions for the challenge images to gradescope under “Project 2 Challenge Submission.”

A Implementation notes

A.1 `torch.nn.Conv2d`

This class implements the 2D convolutional layer we have learned in lecture. Create the layer as follows: `torch.nn.Conv2d(in_channels, out_channels, kernel_size, stride, padding)`. Use default settings for all other arguments. For example `dilation=1` means no dilation is used, and `bias=True` adds a learnable bias term to the neuron.

A.2 the SAME padding

With Pytorch's default setting `padding=0`, if we apply a 3×3 convolutional filter to a 4×4 input image, the output would be 2×2 . As we keep applying convolutional layers in this way, the spatial dimensions will keep decreasing. However, in the early layers of a CNN, we want to preserve as much information about the original input volume so that we can extract those low level features. To do this, we can pad the borders of the input image with zeros in a way such that the output dimension is the **SAME** as the input (assuming unit strides). If the filter size is odd $k = 2m + 1$, then this amount of zero padding on each side is $p = \lfloor k/2 \rfloor = m$. For example, in Figure 6, since the filter size is $k = 3$, the appropriate padding size is $p = \lfloor 3/2 \rfloor = 1$.

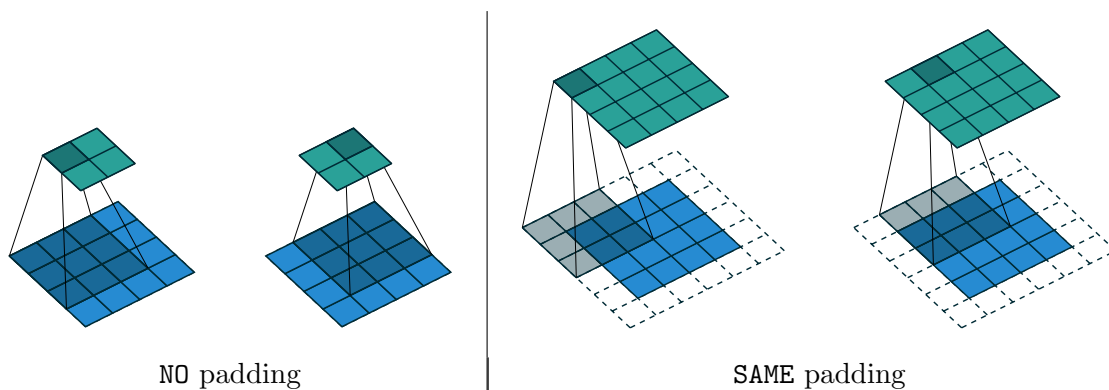


Figure 6: Comparison of padding schemes.

Adapted from: Vincent Dumoulin, Francesco Visin - A guide to convolution arithmetic for deep learning.

Source code: https://github.com/vdumoulin/conv_arithmetic

A.3 `torch.nn.CrossEntropyLoss`

Let $\bar{z}$ be the network output logits and $\hat{y}_c$ the probability that the network assigns to a single input example that belongs to class c . Let D be the number of output classes, and $\bar{y} \in \mathbb{R}^D$ be a vector with a one in the position of the true label and zero otherwise (i.e., if the true label is t , then $\bar{y}_c = 1$ if $t = c$ and $\bar{y}_c = 0$ if $t \neq c$). This is known as *one hot encoding*.

For a multiclass classification problem, we typically apply a soft max activation at the output layer to generate a probability distribution vector. Each entry of this vector can be computed as:

$$\hat{y}_c = \frac{\exp(z_c)}{\sum_j \exp(z_j)}$$

The soft max activation guarantees that each component is in the range between 0 and 1, and that entries sum to 1. We then compare this probability distribution $\hat{y}$ with the ground truth distribution $\bar{y}$ (a one-hot vector), computing the cross entropy loss, $\mathcal{L}$:

$$\mathcal{L}(\bar{y}, \hat{y}) = - \sum_c \bar{y}_c \log \hat{y}_c$$

These two steps can be done at once in PyTorch using the **CrossEntropyLoss** class, which combines the softmax activation with negative log likelihood loss. We don't need to add a separate soft max activation at the output layer. This improves computational efficiency and numerical stability.

B Target Architecture

Note that the dimensions given here match the expected input of Pytorch documentation and may be slightly different from the dimensions used in discussion.

Training Parameters

- Criterion: `torch.nn.CrossEntropyLoss`
- Optimizer: `torch.optim.Adam`
- Learning rate: 10^{-3}
- Patience: 5
- Batch Size: 32

Architecture

Layer 0: Input image

- Output: $3 \times 64 \times 64$

Layer 1: Convolutional Layer 1

- Number of filters: 16
- Filter size: 5×5
- Stride size: 2×2
- Padding: SAME
- Activation: ReLU
- Weight initialization: normally distributed with $\mu = 0.0$, $\sigma^2 = \frac{1}{5 \times 5 \times \text{num_input_channels}}$
- Bias initialization: constant 0.0
- Output: $16 \times 32 \times 32$

Layer 2: Max Pooling Layer

- Kernel size: 2×2
- Stride: 2×2
- Padding: none

Layer 3: Convolutional Layer 2

- Number of filters: 64
- Filter size: 5×5
- Stride size: 2×2
- Padding: SAME
- Activation: ReLU

- Weight initialization: normally distributed with $\mu = 0.0$, $\sigma^2 = \frac{1}{5 \times 5 \times num_input_channels}$
- Bias initialization: constant 0.0
- Output: $64 \times 8 \times 8$

Layer 4: Max Pooling Layer

- Kernel size: 2×2
- Stride: 2×2
- Padding: none

Layer 5: Convolutional Layer 3

- Number of filters: 8
- Filter size: 5×5
- Stride size: 2×2
- Padding: SAME
- Activation: ReLU
- Weight initialization: normally distributed with $\mu = 0.0$, $\sigma^2 = \frac{1}{5 \times 5 \times num_input_channels}$
- Bias initialization: constant 0.0
- Output: $8 \times 2 \times 2$

Layer 6: Fully connected layer 1 (Output layer)

- Input: 32
- Activation: None
- Weight initialization: normally distributed with $\mu = 0.0$, $\sigma^2 = \frac{1}{input_size}$
- Bias initialization: constant 0.0
- Output: 2 (number of classes)

C Source Model Architecture

Note that the dimensions given here match the expected input of Pytorch documentation and may be slightly different from the dimensions used in discussion.

Training Parameters

- Criterion: `torch.nn.CrossEntropyLoss`
- Optimizer: `torch.optim.Adam` with 0.01 weight decay
- Learning rate: 10^{-3}
- Patience: 10
- Batch Size: 64

Architecture

Layer 0: Input image

- Output: $3 \times 64 \times 64$

Layer 1: Convolutional Layer 1

- Number of filters: 16
- Filter size: 5×5
- Stride size: 2×2
- Padding: SAME
- Activation: ReLU
- Weight initialization: normally distributed with $\mu = 0.0$, $\sigma^2 = \frac{1}{5 \times 5 \times num_input_channels}$
- Bias initialization: constant 0.0
- Output: $16 \times 32 \times 32$

Layer 2: Max Pooling Layer

- Kernel size: 2×2
- Stride: 2×2
- Padding: none

Layer 3: Convolutional Layer 2

- Number of filters: 64
- Filter size: 5×5
- Stride size: 2×2
- Padding: SAME
- Activation: ReLU

- Weight initialization: normally distributed with $\mu = 0.0$, $\sigma^2 = \frac{1}{5 \times 5 \times num_input_channels}$
- Bias initialization: constant 0.0
- Output: $64 \times 8 \times 8$

Layer 4: Max Pooling Layer

- Kernel size: 2×2
- Stride: 2×2
- Padding: none

Layer 5: Convolutional Layer 3

- Number of filters: 8
- Filter size: 5×5
- Stride size: 2×2
- Padding: SAME
- Activation: ReLU
- Weight initialization: normally distributed with $\mu = 0.0$, $\sigma^2 = \frac{1}{5 \times 5 \times num_input_channels}$
- Bias initialization: constant 0.0
- Output: $8 \times 2 \times 2$

Layer 6: Fully connected layer 1

- Input: 32
- Activation: None
- Weight initialization: normally distributed with $\mu = 0.0$, $\sigma^2 = \frac{1}{input_size}$
- Bias initialization: constant 0.0
- Output: 8

D ViT Architecture

This provides a more thorough overview of the ViT Architecture and certain design choices, which are not explicitly covered in class. For more information on the architecture, we encourage you to read the original paper by [Dosovitskiy et al., 2021](#).

D.1 Patch Representations

First, our input into a ViT is an image, or rather a sequence of the patches of the image, which are 3-dimensional (channel $\times$ height $\times$ width). A typical transformer encoder only takes in a sequence of vector embeddings, so the first thing we need to do is to project our image patches to a vector embedding. Mathematically, let P be the height and width of our image patch, and let d be the embedding dimension. We first flatten the $3 \times P \times P$ image patches to $3P^2$ -dimensional vectors and then use a linear layer to project to d -dimensional vectors. (Note: we use the same linear layer across all patches.)

D.2 Classification Token

Next, you might notice that there is an extra learnable `[class]` embedding in Figure 5. This is an extra learnable embedding of dimension d . We call this the `[cls]` token embedding.

D.3 Positional Embeddings

Besides token embeddings, we also have positional embeddings. Many people have tried learnable positional embeddings, though in this project we are sticking with the sinusoidal positional embeddings covered in lecture.

D.4 Transformer Encoder

The next part is a transformer encoder. The encoder is composed of transformer blocks. In each block, we have three main parts: [LayerNorm](#), multi-head attention (MHA), and a multilayer perceptron (MLP).

D.4.1 Multi-Head Attention

In MHA, we perform self-attention in each head in parallel, with the query, key, and value matrices W^Q, W^K, W^V (these are essentially linear layers). In the provided ViT implementation, each head has a learned mapping from the input sequence dimensions to a smaller dimensional representation. By having this reduction, MHA has similar runtime to single-headed attention, however each head is able to learn from different subspaces.

After computing the attention in each head, we concatenate our output and finally pass them through another linear layer W_o or `c_proj` in the code (see [Vaswani et al., 2017](#) for more details). This implementation of ViT has each i th head learn separate weight matrices W_i^Q, W_i^K, W_i^V , then concatenate them together after calculating attention scores. However, some implementations have only one of each Q, K, V matrix for the entire transformer, whose output is subsequently split between the attention heads. Note that these two implementations are equivalent.

For this implementation of MHA, the Linear layers representing W^Q, W^K, W^V matrices have biases.

D.4.2 MLP Layer

For the MLP layer, the most common design choice (including in this ViT) is having a 2-layer MLP, where we project our embeddings from d dimensions to $4d$ dimensions in the first layer, then from $4d$ back to d in the second layer.

D.5 ViT MLP Head

After the transformer encoder, we essentially have a sequence of outputs (recall the transformer encoder is seq2seq). Now, we **only** take the output at the first position, i.e. the position corresponding to the [cls] token in the input, and put that in an MLP head. In this project, our MLP head is a single fully connected layer that projects the embedding from d to C , where C denotes the number of classes.

Training Parameters

- Criterion: `torch.nn.CrossEntropyLoss`
- Optimizer: `torch.optim.Adam`
- Learning rate: 10^{-3}
- Batch Size: 32

Architecture

- Number of Patches: 16×16 grid of 256 total patches
- Patch Embedding Dimension: 16
- Number of Attention Heads: 2
- Number of Transformer Blocks: 2